Practical Assignment based on C++ concepts

- **Assignment 1: Employee Record System**

```
#include <iostream>
#include <string>
using namespace std;

class Employee
{
private:
   int empID;
   string name;
   float basicSalary;
   string designation;
   float grossSalary;
   static int empCount;

public:
   Employee(int id, string nm, float salary, string desig)
   {
      empID = id;
      name = nm;
      basicSalary = salary;
      designation = desig;
      grossSalary = 0;
      empCount++;
   }

   void calculateGrossSalary()
   {
      float DA = 0.80f * basicSalary;
```

```
    float HRA = 0.20f * basicSalary;
    grossSalary = basicSalary + DA + HRA;
}

void displaySalaryDetails() const
{
    cout << "Gross Salary (without bonus): " << grossSalary << endl;
}

void displaySalaryDetails(float bonus) const
{
    cout << "Gross Salary (with bonus): " << grossSalary + bonus << endl;
}

void displayDetails() const
{
    cout << "\n--- Employee Details ---\n";
    cout << "ID: " << empID << endl;
    cout << "Name: " << name << endl;
    cout << "Designation: " << designation << endl;
    cout << "Basic Salary: " << basicSalary << endl;
}

float getGrossSalary() const
{
    return grossSalary;
}

static int getEmpCount()
{
    return empCount;
```

```
    }
};

int Employee::empCount = 0;

Employee getHighestPaidEmployee(Employee e1, Employee e2, Employee e3)
{
    if (e1.getGrossSalary() > e2.getGrossSalary() && e1.getGrossSalary() > e3.getGrossSalary())
        return e1;
    else if (e2.getGrossSalary() > e3.getGrossSalary())
        return e2;
    else
        return e3;
}

int main()
{
    Employee emp1(1, "Shreeja", 40000, "Manager");
    Employee emp2(2, "Nandini", 35000, "Developer");
    Employee emp3(3, "Rujuta", 38000, "Analyst");

    emp1.calculateGrossSalary();
    emp2.calculateGrossSalary();
    emp3.calculateGrossSalary();

    emp1.displayDetails();
    emp1.displaySalaryDetails();
    emp1.displaySalaryDetails(2000);

    emp2.displayDetails();
    emp2.displaySalaryDetails();
```

```
    emp2.displaySalaryDetails(1500);

    emp3.displayDetails();
    emp3.displaySalaryDetails();
    emp3.displaySalaryDetails(1800);

    Employee highest = getHighestPaidEmployee(emp1, emp2, emp3);
    cout << "\n** Employee with Highest Gross Salary **";
    highest.displayDetails();
    highest.displaySalaryDetails();

    cout << "\nTotal Employees Created: " << Employee::getEmpCount() << endl;

    return 0;
}
```

- **Assignment 2: Complex Number Operation**

```
#include <iostream>
using namespace std;

class Complex {
private:
   float real, img;

public:
   Complex(float r = 0, float i = 0) {
      real = r;
      img = i;
   }

   void display() const {
      if (img >= 0)
         cout << real << " + " << img << "i";
      else
         cout << real << " - " << -img << "i";
   }

   Complex operator+(const Complex& c) const {
      return Complex(real + c.real, img + c.img);
   }

   Complex operator*(const Complex& c) const {
      float r = real * c.real - img * c.img;
      float i = real * c.img + img * c.real;
      return Complex(r, i);
   }
```

```
    friend Complex operator-(const Complex& c1, const Complex& c2);

    friend bool operator==(const Complex& c1, const Complex& c2);
};

Complex operator-(const Complex& c1, const Complex& c2) {
    return Complex(c1.real - c2.real, c1.img - c2.img);
}

bool operator==(const Complex& c1, const Complex& c2) {
    return (c1.real == c2.real) && (c1.img == c2.img);
}

int main() {
    float r1, i1, r2, i2;

    cout << "Enter real and imaginary part of first complex number: ";
    cin >> r1 >> i1;
    cout << "Enter real and imaginary part of second complex number: ";
    cin >> r2 >> i2;

    Complex c1(r1, i1), c2(r2, i2);

    Complex sum = c1 + c2;
    Complex diff = c1 - c2;
    Complex prod = c1 * c2;
    bool equal = (c1 == c2);

    cout << "\nFirst Complex Number: ";
    c1.display();
```

```
    cout << "\nSecond Complex Number: ";
    c2.display();

    cout << "\n\nSum: ";
    sum.display();

    cout << "\nDifference: ";
    diff.display();

    cout << "\nProduct: ";
    prod.display();

    cout << "\nEquality Check: ";
    if(equal)
    {
        cout << "Equal";
    }
    else
    {
        cout << "Not Equal";
    }

    return 0;
}
```

- **Assignment 3: Student Result Management using Inheritance**

```
#include <iostream>
#include <string>
using namespace std;

class Student
{
   public:
   int rollNo;
   string name;
   void inputDetails()
   {
      cout << "Enter your Roll no.: ";
      cin >> rollNo;
      cout << "Enter your name: ";
      cin >> name;
   }

   void displayDetails()
   {
      cout << "Roll no.: " << rollNo << "\nName: " << name << endl;
   }
};

class Marks: public Student
{
   public:
   int subject1, subject2, subject3;
   void inputMarks()
   {
      cout << "Enter marks for 3 Subjects:";
      cin >> subject1 >> subject2 >> subject3;
   }

   void displayMarks()
   {
      cout << "Subject 1: " << subject1 << "\nSubject 2: " << subject2 << "\nSubject 3: " <<
subject3 << endl;
   }
};

class Result: public Marks
{
   public:
   float total, percentage;
   char grade;
   void calculate()
```

```
    {
        total=subject1+subject2+subject3;
        percentage=total/3.0;
        if(percentage>=75)
        {
            grade = 'A';
        }
        else if(percentage>=60)
        {
            grade = 'B';
        }
        else if(percentage>=50)
        {
            grade = 'C';
        }
        else
        {
            grade = 'D';
        }
    }
    void displayResult()
    {
        displayDetails();
        displayMarks();
        cout << "Total: " << total << "\nPercentage: " << percentage <<"%\nGrade: " << grade <<
endl;
    }
};

int main()
{
    Result students[5];
    for(int i=0;i<5;i++)
    {
        cout << "\nEnter details for Student" << i+1 << ":\n";
        students[i].inputDetails();
        students[i].inputMarks();
        students[i].calculate();
    }
    cout << "\n--- Student Result ---\n";
    for(int i=0;i<5;i++)
    {
        cout << "\nStudent" << i+1 << ":\n";
        students[i].displayResult();
    }

return 0;
}
```

- **Assignment 4: Book Library with Nested Class**

```
#include <iostream>
#include <string>
using namespace std;

class Library
{
public:
   class Book
   {
      string title, author, ISBN;
      float price;
      int copies;

   public:
      Book() {
         title = author = ISBN = "";
         price = 0.0;
         copies = 0;
      }

      void input() {
         cout << "Enter title, author, ISBN, price, copies:\n";
         cin >> title >> author >> ISBN >> price >> copies;
      }

      void display() const {
         cout << "Title: " << title << "\nAuthor: " << author
            << "\nISBN: " << ISBN << "\nPrice: " << price
            << "\nCopies: " << copies << endl;
      }
```

```
    string getISBN() const {
        return ISBN;
    }

    string getTitle() const {
        return title;
    }
};

Book books[10];
int count;

public:
Library()
{
    count = 0;
}

void addBook()
{
    if (count < 10) {
        books[count].input();
        count++;
    } else {
        cout << "Library full.\n";
    }
}

void searchByISBN(const string &isbn) const
{
```

```
    for (int i = 0; i < count; i++)
    {
        if (books[i].getISBN() == isbn)
        {
            books[i].display();
            return;
        }
    }
    cout << "Book not found.\n";
}

void searchByTitle(const string &title) const
{
    for (int i = 0; i < count; i++)
    {
        if (books[i].getTitle() == title)
        {
            books[i].display();
            return;
        }
    }
    cout << "Book not found.\n";
}

int factorial(int n) const
{
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

void showTotalBooksFactorial() const
```

```
    {
        cout << "Factorial of total books: " << factorial(count) << endl;
    }
};

int main(int argc, char *argv[])
{
    Library lib;

    int n;
    cout << "How many books to add? ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        lib.addBook();
    }

    string isbn, title;
    cout << "\nEnter ISBN to search: ";
    cin >> isbn;
    lib.searchByISBN(isbn);

    cout << "\nEnter Title to search: ";
    cin >> title;
    lib.searchByTitle(title);

    lib.showTotalBooksFactorial();

    return 0;
}
```

- **Assignment 5: Bank Account System with Method and Constructor Overloading**

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount
{
   int accountNumber;
   string accountHolder;
   float balance;
   string type;

   static int activeAccounts;

public:
   BankAccount()
   {
      accountNumber = 0;
      accountHolder = "";
      balance = 0.0;
      type = "N/A";
      activeAccounts++;
   }

   BankAccount(int accNo, string holder, float bal, string accType)
   {
      accountNumber = accNo;
      accountHolder = holder;
      balance = bal;
      type = accType;
      activeAccounts++;
```

```
}

void deposit(float amount)
{
   balance += amount;
}

void deposit()
{
   float amount;
   cout << "Enter amount to deposit: ";
   cin >> amount;
   balance += amount;
}

void withdraw(float amount)
{
   if (amount <= balance)
      balance -= amount;
   else
      cout << "Insufficient balance\n";
}

void withdraw()
{
   float amount;
   cout << "Enter amount to withdraw: ";
   cin >> amount;
   if (amount <= balance)
      balance -= amount;
   else
```

```
        cout << "Insufficient balance\n";
}

void display() const
{
    cout << "Account No: " << accountNumber << "\nHolder: " << accountHolder
         << "\nBalance: " << balance << "\nType: " << type << endl;
}

static int getActiveAccounts()
{
    return activeAccounts;
}

BankAccount transfer(BankAccount &receiver, float amount)
{
    if (amount <= balance)
    {
        balance -= amount;
        receiver.balance += amount;
    }
    else
    {
        cout << "Transfer failed. Insufficient balance.\n";
    }
    if (balance > receiver.balance)
    {
        return *this;
    }
    else
    {
```

```
            return receiver;
        }
    }

    float getBalance() const
    {
        return balance;
    }
};

int BankAccount::activeAccounts = 0;

int main() {
    BankAccount a1(1, "Shivani", 7000, "Savings");
    BankAccount a2(2, "Rishima", 4500, "Current");

    a1.deposit(1500);
    a2.withdraw(1000);

    BankAccount richer = a1.transfer(a2, 2000);

    cout << "\nAccount 1:\n";
    a1.display();

    cout << "\nAccount 2:\n";
    a2.display();

    cout << "\nAccount with higher balance:\n";
    richer.display();

    cout << "\nTotal Active Accounts: " << BankAccount::getActiveAccounts() << endl;
```

```
    return 0;
}
```